

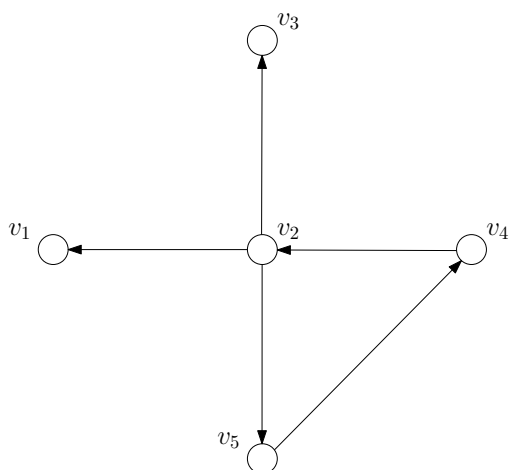
GRAFOVÉ ALGORITMY

1 Grafy a jejich reprezentace

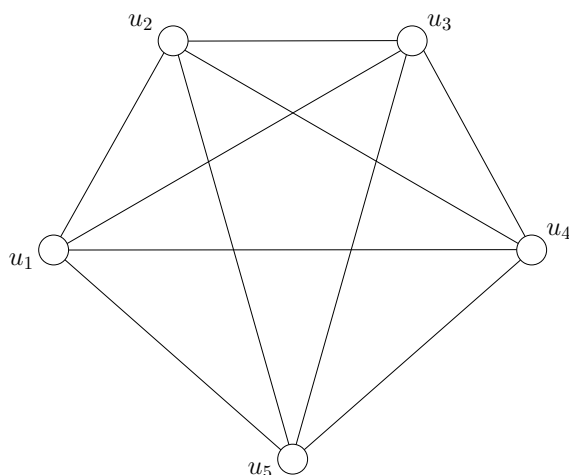
Grafy jsou základním stavebním kamenem pro mnoho úloh informatice. Lze jimi obecně reprezentovat mnoho situací, v nichž jsou klíčové určité prvky a jejich vzájemné vztahy. Např. propojení zařízení (router, switch, hub či koncový uživatel) v počítačové síti, schéma zapojení elektrického obvodu, mapa města nebo např. v chemii vazby mezi atomy. Jejich matematická podstata je však velice jednoduchá – *vrcholy* propojené *hranami*.

Definice 1.1 (Graf). Grafem G nazveme uspořádanou dvojici (V, E) , kde V je množina *vrcholů* (nebo také *uzlů*) a E množina *hran*, přičemž pokud

- $E \subseteq \{\{u, v\} \mid u, v \in V\}$, pak G nazýváme *neorientovaným* grafem (tj. po hraně lze pohybovat v obou směrech).
- $E \subseteq \{(u, v) \mid u, v \in V\}$, pak G nazýváme *orientovaným* grafem (tj. po hranách se lze pohybovat pouze v jednom směru).



(a) Příklad orientovaného grafu.

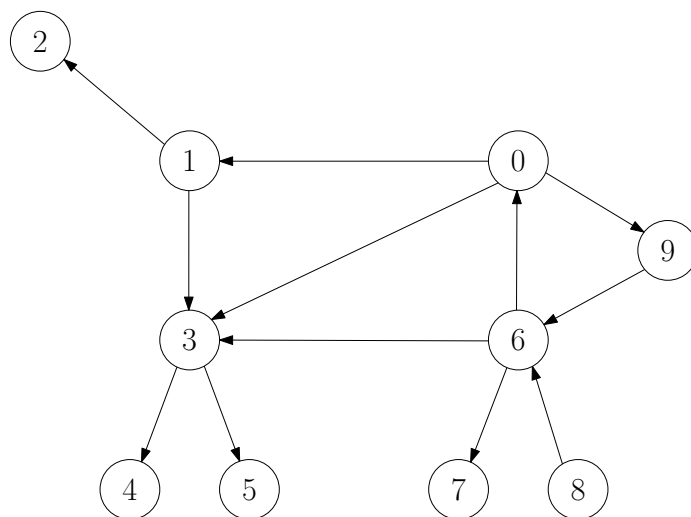


(b) Příklad neorientovaného grafu.

Podstatnou otázkou u grafů jsou způsoby jejich reprezentace, neboť při programování různé způsoby různě efektivní a je dobré si mezi nimi umět vybrat. Pracujme tedy s obecným grafem $G = (V, E)$, přičemž vrcholy grafu si pro jednoduchost označíme přirozenými čísly $1, 2, \dots, n$, tzn. $V = \{1, 2, \dots, n\}$.

- **Matice sousednosti.** První způsob, jak graf reprezentovat, je pomocí matice. V tomto případě graf G popisuje matice $n \times n$, kde na pozici ij je 1, pokud z vrcholu i do vrcholu j vede hrana, jinak 0.

Příklad 1.2 (Prásátkový graf). Orientovaný graf G (viz obrázek 2) a jeho matice sousednosti A .

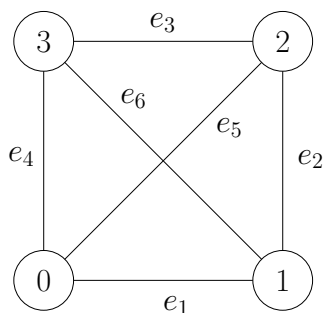


Obrázek 2: Prasátkový graf.

$$A = \begin{pmatrix} 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{pmatrix}$$

- **Matice incidence.** Řádky jsou indexovány vrcholy a sloupce jednotlivými hranami. Na místě ij je 1, pokud hrana vede *do* vrcholu i , nebo -1 , pokud z něj vychází, jinak 0. Pro neorientované grafy píšeme vždy 1, pokud daná hrana vede *do/z* vrcholu.

Příklad 1.3 (Úplný graf K_4). Graf G má 4 vrcholy a celkem 6 hran (viz obrázek 3) a její matice incidence I . Řádky představují (shora dolů) čísla vrcholů 0, 1, 2, 3 a sloupce (zleva doprava) jednotlivé hrany $e_1, \dots, e_6$.

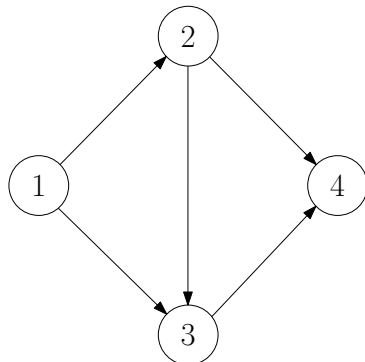


Obrázek 3: Úplný graf K_4 .

$$I = \begin{pmatrix} 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \end{pmatrix}$$

- **Seznam sousedů.** Jedná se pravděpodobně o nejpoužívanější variantu reprezentace grafů. Pro každý vrchol $v \in V$ si jednoduše pamatujeme seznam¹ $S[v]$ všech vrcholů, do kterých přímo vede hrana z vrcholu v .

Příklad 1.4. Graf G s vrcholy $1, \dots, 4$ a jejich seznamy sousedů S (viz obrázek 4).



$$\begin{array}{ll} S[1] = \{2, 3\} & S[3] = \{4\} \\ S[2] = \{3, 4\} & S[4] = \emptyset \end{array}$$

Obrázek 4: Graf G a seznamy sousedů pro jeho každý vrchol.

Tím jsme si ukázali základní možnosti reprezentace grafů, avšak zatím jsme si je mezi sebou nijak neporovnali. Pojďme se tedy podívat na paměťové nároky daných variant. Označme si počet vrcholů grafu $|V| = n$ a celkový počet hran $|E| = m$.

- *Matice sousednosti* spotřebuje paměť $\mathcal{O}(n^2)$, neboť potřebujeme uložit matici velikosti $n \times n$. Je tak vhodná především pro husté grafy, tedy grafy s velkým počtem hran.
- *Matice incidence* využije $\mathcal{O}(nm)$ paměťových buněk. Tento způsob reprezentace je vhodný především pro matematickou práci s grafy, avšak při programování se moc nevyužívá, neboť obsahuje spoustu nadbytečných informací.
- *Reprezentace pomocí seznamů sousedů* je obecně hodně využívána. Řadově spotřebuje $\mathcal{O}(n + m)$ paměti² a je tak dobrou alternativou oproti matici sousednosti v případech, kdy naopak daný graf je řídký, tedy neobsahuje příliš mnoho hran (např. strom).

2 Stromy

Stromy tvoří speciální oblast v teorii grafů. V informatice se jedná silně dominující typ grafu a bylo by proto dobré mu věnovat zvláštní sekci, kde si připomeneme některé základní typy.

Definice 2.1 (Strom). Graf $G = (V, E)$ je strom, pokud

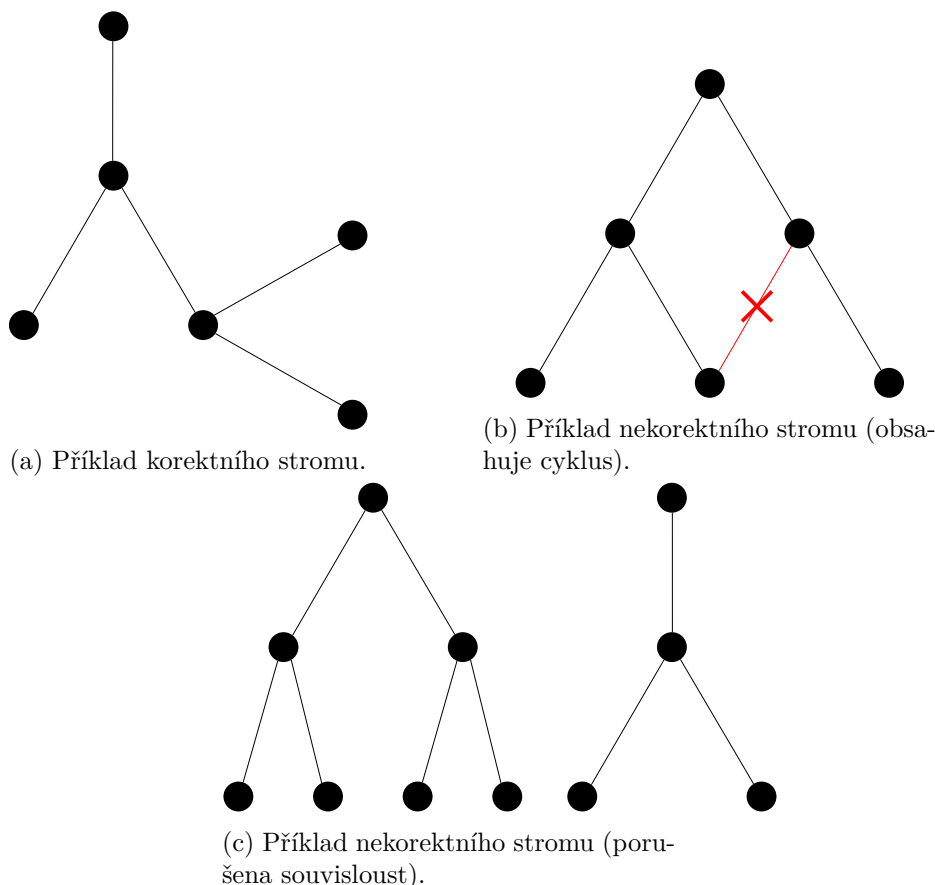
- je souvislý³
- a neobsahuje kružnici.

Je dobré si uvědomit, že mezi každou dvojicí vrcholů ve stromě vede **právě jedna** cesta. Zároveň z definice stromu také plyne, že odebráním libovolné hrany se již nutně poruší souvislost (minimálně přestane existovat cesta mezi koncovými vrcholy).

¹Formálně vzato se spíše jedná množinu.

²Odvození zatím necháme stranou, avšak jednoduchým důvodem je fakt, že si musíme v paměti uchovat celkem n vrcholů a pro každý z nich si musíme pamatovat všechny hrany, které z nich vychází. Blíže se na tento argument podíváme v odvození časové složitosti algoritmu BFS, viz 3.1.

³Mezi každou dvojicí vrcholů vede cesta.



Poslední příklad 5c sice není z definice strom, avšak čtenář by mohl namítnout, že jednotlivé “části” daného grafu tvoří stromy. Tomuto typu grafů se říká (celkem pochopitelně) *lesy*, neboť jejich *komponenty souvislosti*⁴ tvoří stromy (viz obrázek 6).

Definice 2.2 (Les). Graf $G = (V, E)$ je les, pokud každá jeho komponenta souvislosti je strom.

Ekvivalentně můžeme les definovat jako *sjednocení stromů*. Jako poslední si uvedme speciální typ stromu, se kterým budeme dále nejčastěji pracovat, a to tzv. *binární strom*. Blíže se s ním setkáme v sekci 5, kdy budeme rozebírat tzv. *binární haldu*.

Definice 2.3 (Binární strom). Strom $T = (V, E)$ nazveme binární, pokud

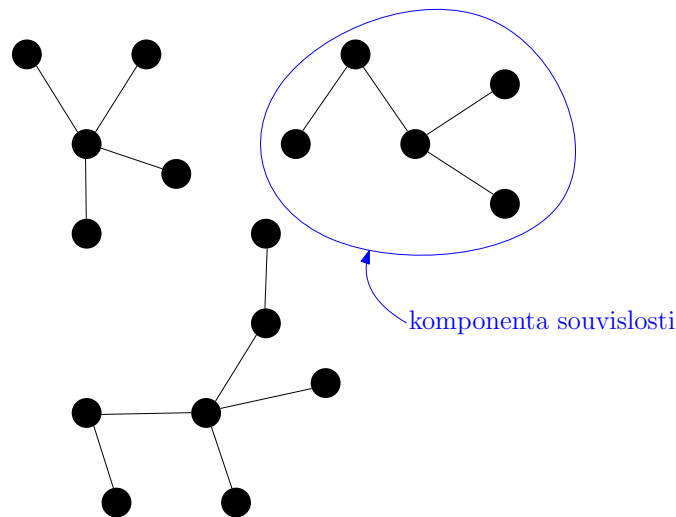
- (i) je zakořeněný
- (ii) a každý vrchol má nejvýše dva syny.

Jeden z vrcholů stromu T je označen jako *kořen*. Typicky jej zakreslujeme nahoru nad všechny ostatní vrcholy a syny každého vrcholu umísťujeme vždy níže oba na stejnou úroveň (viz obrázek 7).

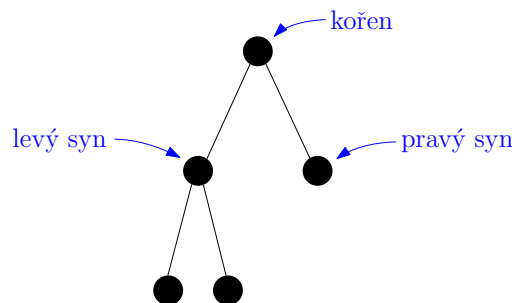
3 Prohledávání do šířky

Jednou ze základních úloh je procházení grafu z určitého vrcholu a zjištění dosažitelnosti ostatních vrcholů. Nejednodušším algoritmem v tomto ohledu je tzv. *prohledávání do šířky* (angl. *breadth-first*

⁴Intuitivně se jedná o část grafu, kde mezi každými dvěma vrcholy vede cesta. Formální zavedení termínu se provádí přes *binární relaci*. Pro zájemce: [https://en.wikipedia.org/wiki/Component_\(graph_theory\)](https://en.wikipedia.org/wiki/Component_(graph_theory)).



Obrázek 6: Příklad lesa



Obrázek 7: Příklad binárního stromu

search, zkráceně BFS). Jeho základní princip spočívá v postupném objevování následníků již nalezených vrcholů. Na počátku dostaneme graf $G = (V, E)$ a nějaký počáteční vrchol $v_0 \in V$. Postupně objevíme všechny sousedy vrcholu v_0 , poté všechny sousedy těchto nalezených sousedů, atd. Na BFS lze nahlížet tak, že do počátečního vrcholu nalijeme vodu a sledujeme, jak postupuje vzniklá vlna.

Pro každý vrchol si budeme uchovávat jeho *stav*.

- *Nenalezený* – vrchol jsme ještě během výpočtu neviděli.
- *Otevřený* – vrchol jsme viděli, ale ještě nejsme neprozkoumali všechny jeho sousedy.
- *Uzavřený* – vrchol jsme prozkoumali společně se všemi jeho sousedy a dál se jím již netřeba zabývat.

Na počátku začneme s jedním otevřeným vrcholem a to v_0 (zde začínáme). Po prozkoumání všech sousedních vrcholů se jejich stav změní na uzavřený a počáteční vrchol v_0 se uzavře. Obdobně pokračujeme pro nově otevřené vrcholy. Pokud by náhodou mezi dvojicí otevřených vrcholů existovala hrana, pak si sousedního vrcholu všimnat nebudeme, neboť byl již otevřen. Pro každý vrchol se ještě dodatečně můžeme uchovávat informaci, jak daleko se nachází od v_0 , co do počtu hran ležících na cestě.

BFS rozděluje vrcholy do vrstev podle toho, v jaké vzdálenosti od počátečního vrcholu se nachází (viz obrázek 8). Nejdříve jsou prozkoumány vrcholy ve vzdálenosti 0 (tj. pouze v_0), poté ve vzdálenostech 1, 2, ... Z toho vyplývá, že kdykoliv při otevírání libovolného vrcholu v nastavujeme hodnotu $D(v)$, bude tato hodnota vždy odpovídat délce nejkratší cesty z v_0 do v . Tato hodnota bude však nastavena pouze u těch vrcholů, které jsou z v_0 dosažitelné (ostatní vrcholy zůstanou ve stavu nenalezený).

Algoritmus 3.1: BFS (prohledávání do šířky)

Vstup: Graf $G = (V, E)$ a počáteční vrchol $v_0 \in V$
// Inicializace
1 **foreach** vrchol $v \in V$ **do**
2 $stav(v) \leftarrow \text{nenalezený}$
3 $D(v) \leftarrow \infty$
4 $stav(v_0) \leftarrow \text{otevřený}$
5 $D(v_0) \leftarrow 0$
6 Založ frontu Q a přidej do ní vrchol v_0
7 **while** fronta Q je neprázdná **do**
8 $v \leftarrow$ první vrchol ve frontě Q , který z ní odebereme
9 // Prozkoumáváme všechny dosud neobjevené sousedy
10 **foreach** sousední vrchol w vrcholu v **do**
11 **if** $stav(w) = \text{nenalezený}$ **then**
12 $stav(w) \leftarrow \text{otevřený}$
13 $D(w) \leftarrow D(v) + 1$
14 Přidej w do fronty Q
15 $stav(v) \leftarrow \text{uzavřený}$
Výstup: Seznam vzdáleností D

Zbývá prozkoumat časovou a paměťovou složitost BFS. Označme si počet vrcholů grafu G na vstupu n a počet jeho hran m .

Věta 3.1 (Složitost BFS). Algoritmus BFS doběhne v čase $\mathcal{O}(n + m)$ a spotřebuje paměť $\mathcal{O}(n + m)$.

Důkaz. Inicializace potrvá $\mathcal{O}(n)$, neboť cyklus iteruje přes všechny vrcholy. Vnější cyklus provede maximálně n iterací, protože každý z vrcholů uzavřeme nejvýše jednou, tj. $\mathcal{O}(n)$.

S vnitřním cyklem je to trochu složitější, protože jeho počet iterací závisí na tom, který z vrcholů otevíráme (resp. na počtu jeho sousedů). To znamená, že pokud si označíme d_i počet sousedů vrcholu i , pak celkový počet iterací vnitřního cyklu přes všechny vrcholy bude $\sum_i d_i$. Lze si ovšem všimnout jedné užitečné věci. Pokaždé, když prozkoumáváme sousední vrchol w nějakého vrcholu v , mohou nastat dva případy podle toho, jestli je G orientovaný graf, nebo neorientovaný.

- (i) Graf G je neorientovaný. Pak hranu, která spojuje v a w prozkoumáme právě *dvakrát* (jednou z vrcholu v a podruhé z vrcholu w). Každá hrana se tak započítá dvakrát, tzn. vnitřní cyklus se celkově provede max $2m$ -krát (po všech iteracích vnějšího cyklu).
- (ii) Graf G je orientovaný. Pak se hrana započítá pouze jednou, a to z vrcholu, z něhož vede. Celkově se vnitřní cyklus provede m -krát.

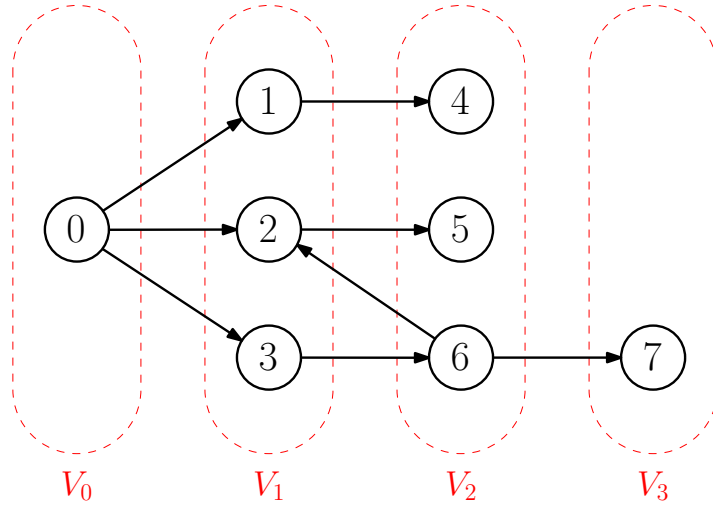
V prvním případě tak pro časovou složitost bude platit

$$\mathcal{O}(n + \sum_i d_i) = \mathcal{O}(n + 2m) \stackrel{*}{=} \mathcal{O}(n + m).$$

(V kroku označeném $*$ zanedbáváme konstantu, neboť pracujeme s $\mathcal{O}$ -notací.)

V druhém případě dojdeme ke stejnému výsledku, akorát zmíněná suma bude, kvůli orientaci hran, rovna přesně počtu hran, tj.

$$\mathcal{O}(n + \sum_i d_i) = \mathcal{O}(n + m).$$

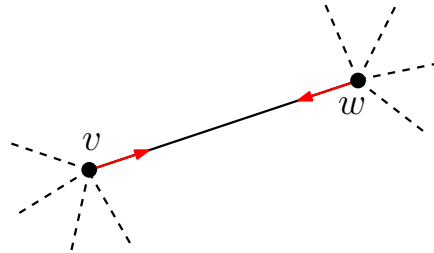


Obrázek 8: Vrcholy rozdělené do vrstev podle průběhu BFS.

Nyní k prostorové složitosti algoritmu. Na reprezentaci grafu (např. pomocí seznamu sousedů) spotřebujeme paměť $\mathcal{O}(n + m)$ (n vrcholů, m hran)⁵. Dále si uchováváme vrcholy ve frontě, v níž může být v jednu chvíli maximálně n vrcholů, tj. $\mathcal{O}(n)$, a k tomu máme seznam *stav*, kde je uloženo $\mathcal{O}(n)$ vrcholů. Celkově spotřebujeme paměti

$$\mathcal{O}(n + m) + \mathcal{O}(n) + \mathcal{O}(n) = \mathcal{O}(n + m).$$

□



Obrázek 9: Znázornění situace v důkazu části (i) věty 3.1.

4 Prohledávání do hloubky

Na podobném přístupu, jako BFS, je založeno tzv. *prohledávání do hloubky* (anglicky *depth-first search*, zkráceně DFS). Vrcholy však tentokrát budeme zpracovávat rekurzivně. Pokaždé, když budeme otevírat nový vrchol, se rekurzivně zavoláme všechny jeho sousední vrcholy, u nichž opakujeme stejnou proceduru. Po prozkoumání všech sousedů daný vrchol uzavřeme. Stejně jako BFS si tak budeme uchovávat pole *stavů* pro jednotlivé vrcholy.

Věta 4.1 (Složitost DFS). Algoritmus DFS doběhne v čase $\mathcal{O}(n + m)$ a spotřebuje paměť $\mathcal{O}(n + m)$.

Důkaz. Algoritmus DFS se oproti BFS liší pouze v pořadí, v jakém pořadí dosažitelné vrcholy zpracovává, to však nemá na časovou složitost žádný vliv. Argument pro její odvození je tak stejný jako u BFS, viz věta 3.1 v minulé sekci.

⁵Resp. každá hrana je v neorientovaných grafech započítána dvakrát, protože každý vrchol obsahuje v seznamu svých sousedů vždy „protějščí“ vrchol (stejný argument, jako u odvozování časové složitosti BFS).

Algoritmus 4.1: DFS (prohledávání do hloubky)

Vstup: Graf $G = (V, E)$ a počáteční vrchol $v_0 \in V$
// Inicializace
1 **foreach** vrchol $v \in V$ **do**
2 $stav(v) \leftarrow \text{nenalezený}$
3 Zavolej DFS2(v_0)
 // Rekurzivní volání na sousední vrcholy
4 **Funkce** DFS2(vrchol $v \in V$)
5 $stav(v) \leftarrow \text{otevřený}$
6 **foreach** sousední vrchol w vrcholu v **do**
7 **if** $stav(w) = \text{nenalezený}$ **then**
8 Zavolej DFS2(w)
9 $stav(v) \leftarrow \text{uzavřený}$

V paměti si musíme uchovávat reprezentaci grafu, to zabere $\mathcal{O}(n+m)$ paměti (opět např. pomocí seznamu sousedů) a máme seznam $stav$ s n prvky (vrcholy). Zároveň si při volání DFS2 musíme na zásobník rekurze ukládat jednotlivé *aktivační záznamy*. Protože vrcholů je v grafu n , pak na zásobníku rekurze bude v jednu chvíli maximálně n záznamů, tj. $\mathcal{O}(n)$. Celkově spotřebujeme $\mathcal{O}(n+m) + \mathcal{O}(n) + \mathcal{O}(n) = \mathcal{O}(n+m)$ paměti. $\square$



Je dobré si uvědomit, že byť algoritmy BFS a DFS mají stejnou časovou a prostorovou složitost, nelze zcela rovnocenně použít na stejné typy úloh. Např. BFS se hodí pro hledání nejkratší cesty *v neohodnoceném grafu* (pro ohodnocené grafy se používá např. Dijkstrův algoritmus, viz sekce 6), kdežto DFS se více hodí na prohledávání stavového prostoru, neboť typicky nezabere tolik paměti.



Cesta, kterou se DFS dostane do libovolného vrcholu v , nemusí být nutně nejkratší (silně závisí na pořadí, v jakém procházíme sousedy jednotlivých vrcholů).

5 Binární halda

Uvedme motivační příklad na úvod. Mějme seznam čísel, z něhož chceme (pokud možno co nejrychleji) vybrat minimální/maximální hodnotu. Pro maximum bychom sestavili následující jednoduchý algoritmus.

Časová složitost bude zjevně $\mathcal{O}(n)$, neboť algoritmus prochází všech n prvků. Takovou úlohu lze však řešit rychleji, pokud si prvky vhodně uspořádáme. K tomu můžeme použít tzv. *haldu*.

Definice 5.1 (Minimová binární halda). Minimová binární halda je datová struktura tvaru binárního stromu, kde v každém vrcholu je uložena *právě jedna* hodnota (tzv. *klíč*, pro vrchol v budeme značit jeho klíč $k(v)$) a navíc platí:

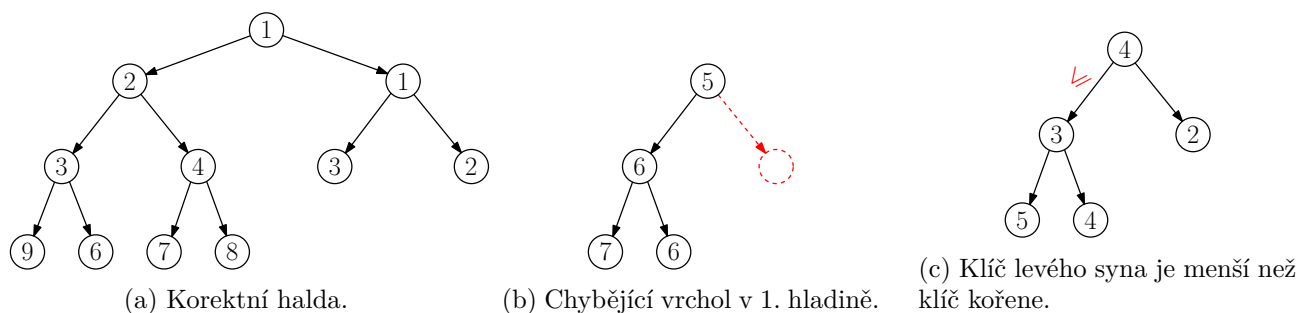
- (i) každá hladina je *plně obsazena*, kromě poslední, přičemž hladiny jsou zaplněny *zleva*
- (ii) a je-li v libovolný vrchol a s jeho syn, pak $k(v) \leq k(s)$.

Algoritmus 5.1: Vyhledání maximální hodnoty v seznamu (MAX)

Vstup: Seznam čísel $x_1, x_2, \dots, x_n$

```
1  $m \leftarrow x_1$ 
2 for  $i = 1, 2, \dots, x_n$  do
3   if  $x_i > m$  then
4      $m \leftarrow x_i$ 
5 return  $m$ 
```

Výstup: Maximální hodnota seznamu m



Obrázek 10: Příklady korektních a nekorektních hald.

Zde se nyní na chvíli pozastavme. Podmínka (ii) v definici binární haldy 5.1 má za následek totiž velmi příjemnou vlastnost, když se podíváme, kde se v haldě nachází minimum. Pokud se budeme pohybovat od kořene směrem „dolů“, hodnoty ve vrcholech se budou pouze zvětšovat, neboť klíče synů musí mít vždy stejnou nebo větší hodnotu než klíč v rodiči. Minimum se tak vždy nachází *v kořeni stromu*, což znamená, že zjištění minima tak můžeme provést v *konstantním čase* $\mathcal{O}(1)$.



Pokud bychom chtěli naopak *maximovou binární haldy*, bude definice 5.1 vypadat obdobně, akorát v podmínce (ii) bude obrácená nerovnost, tj. $k(v) \geq k(s)$ (klíč v rodiči má vždy stejnou nebo vyšší hodnotu než klíče v jeho synech). Maximum se bude opět nacházet v kořeni haldy.

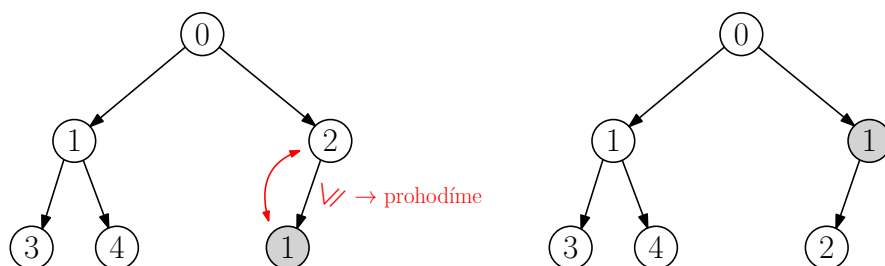
Je však více operací, které bychom rádi s haldou prováděli. Vypišme si všechny, které nás budou zajímat.

- $\text{INSERT}(x)$ – *vložení* nového vrcholu s klíčem x do haldy.
- $\text{MIN}()$ – *nalezení* vrcholu s nejmenším klíčem (už jsme zmínili).
- $\text{EXTRACTMIN}()$ – *odebrání* vrcholu s nejmenším klíčem.
- $\text{INCREASE}(v)$ – *zvýšení* hodnoty klíče ve zvoleném vrcholu v .
- $\text{DECREASE}(v)$ – *snížení* hodnoty klíče ve zvoleném vrcholu v .

Podívejme se postupně na jednotlivé operace a jejich realizaci. S dovolením si zde nyní odpustíme zápis pomocí pseudokódu a pro jednoduchost si dané operace pouze slovně vysvětlíme. Pro následné odvození časové složitosti nám to bude stačit.

5.1 Vkládání nového vrcholu

Při vkládání nového vrcholu (INSERT) se nejdříve potřebujeme vypořádat s tím, kam vrchol v haldě umístíme. S tím nám ale pomůže podmínka (i) v definici binární haldy. Všechny hladiny stromu jsou vždy plně obsazené, kromě poslední, která se všechny vrcholy nachází vlevo. To znamená, že nový vrchol můžeme umístit na první volnou pozici vlevo na poslední hladině. To však nemůžeme provést zcela beztestně, neboť nemáme nijak zaručeno, že bude splněna podmínka (ii). Klíč v novém vrcholu může mít hodnotu *ostře menší* než jeho rodič. Haldu „opravíme“ postupným prohazováním vrcholu s jeho rodičem, dokud nebude podmínka splněna (viz obrázek 11). Postup můžeme zapsat zkráceně ve třech



Obrázek 11: Vkládání nového vrcholu do haldy.

bodech takto:

- Vložíme nový vrchol s klíčem x na první volnou pozici zleva na poslední hladině.
- pokud je klíč rodiče větší než x , prohodíme vrcholy (resp. jejich klíče)⁶.
- Opakujeme druhý krok.

5.2 Odstranění minima

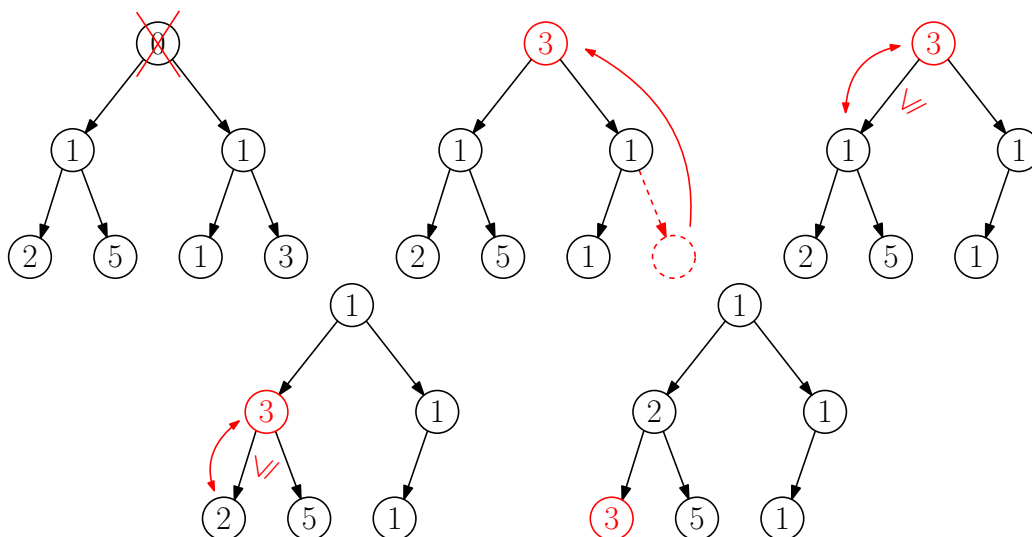
Co se týče získání minimálního klíče v haldě, jedná se o velmi triviální operaci. Zkrátka „přečteme“ klíč v kořeni a jsme hotovi. Ovšem pokud bychom chtěli minimum (EXTRACTMIN) ze stromu odstranit, bude to již o něco složitější. Kořen ze stromu nemůžeme „jen tak“ smazat, jeho pozici musí zastoupit jiný vrchol. Abychom neporušili vlastnosti haldy, nahradíme odstraněný kořen *nejlevějším vrcholem na poslední hladině*. Tím jsme jistě neporušili vlastnost (i), neboť poslední hladina, jako jediná, nemusí být plně obsazená.

Nyní však (stejně jako u vkládání vrcholu) nastává problém s druhou podmínkou, neboť touto operací jsme ji mohli porušit. Provedeme obdobný proces, jako při vkládání vrcholu do haldy, ale vrchol nyní bude „probublávat“ směrem dolů. Podíváme se na syny daného vrcholu a pokud klíč některého z nich je menší než klíč nového kořene, pak jej s ním prohodíme (v případě, že klíče obou synů jsou menší, než klíč kořene, prohodíme s menším z nich). V bodech můžeme zapsat celou operaci následovně:

- Smažeme kořen a nahradíme jej nejlevějším vrcholem na poslední hladině.
- Tento vrchol (resp. jeho klíč) porovnáme s jeho syny a případně prohodíme s tím, jehož klíč je menší.
- Opakujeme druhý krok.

Operacím, které jsme prezentovali při vkládání vrcholu, resp. odebírání minima z haldy, kdy vrchol „probublával“ nahoru, resp. dolů, se někdy nazývají BUBBLEUP a BUBBLEDOWN. Ještě se nám budou

⁶Vrchol v podstatě takto „probublá“ do správné pozice ve stromě (příp. až do pozice kořene).



Obrázek 12: Odebírání vrcholu (kořene) s minimálním klíčem.

hodit u operací INCREASE a DECREASE.

5.3 Zvýšení a snížení hodnoty klíče ve vrcholu

Poslední dvojice operací, která se nám bude hodit, je zvýšení (INCREASE) a snížení (DECREASE) hodnoty klíče ve vybraném vrcholu. K tomu potřebujeme akorát vyřešit, jak budeme k vrcholům přistupovat. Podle klíče vyhledávat neumíme, ale můžeme si zavést např. pomocný slovník, který budeme „adresovat“ pomocí příslušného vrcholu a vrácenou hodnotou bude klíč ve vrcholu. Takto spotřebujeme navíc pouze $\mathcal{O}(n)$ paměti, kde n je počet vrcholů v haldě.

Realizace samotných operací je již jednoduchá. Pokud provedeme INCREASE klíče v určitém vrcholu, pak z důvodu, že klíče v synech vrcholu jsou stejné nebo větší, se halda může pokazit pouze „směrem dolů“. Tedy provedeme nám již známé prohazování vrcholu s jeho syny, dokud nebude halda opravena (BUBBLEDOWN), stejně jako v případě odebírání kořene (EXTRACTMIN).

Operaci DECREASE provedeme zcela stejně, akorát nyní se halda bude kazit „směrem nahoru“, takže provedeme BUBBLEUP, jako při vkládání nového vrcholu (INSERT).



V případě *maximové binární haldy* by operace vypadaly zcela stejně, akorát při operaci INCREASE budeme opravovat haldu směrem nahoru a u DECREASE směrem dolů.

5.4 Časové složitosti operací

Pojďme si nyní rozebrat časové složitosti zmíněných operací INSERT, MIN, EXTRACTMIN, INCREASE a DECREASE a podívat se, jak moc jsme si pomohli oproti práci s polem (seznamem). Pokud se pozorně podíváme na operace popsané výše, je zjevné, že nejvíce bude naše operace zpomalovat ono „proublávání“ vrcholu při opravování haldy (vyjma operace MIN, kde jen přečteme klíč v kořeni), tj. BUBBLEUP a BUBBLEDOWN. Na nich bude celková časová složitost záviset.



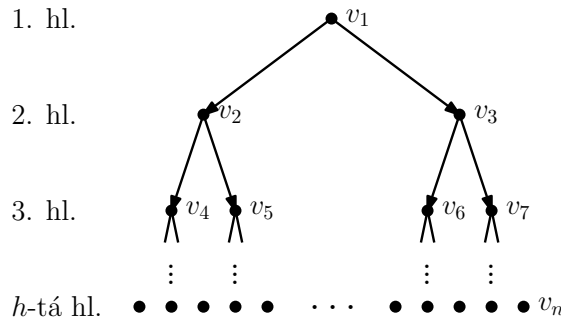
Všimněme si, že po každém prohození vrcholů při BUBBLEUP nebo BUBBLEDOWN se vrchol posune o jednu hladinu výše/níže.

Víme tak, že při opravování haldy se vrchol může posunout *maximálně o počet hladin*, který má daná halda. Časová složitost daných operací bude tak závislá na jejich počtu. Tím jsme se dostali o trochu blíže řešení, ale rádi bychom věděli, jak se bude dobře trvání daných operací měnit v závislosti na *počtu vrcholů v haldě*, nikoliv počtu hladin. Mezi těmito proměnnými však existuje hezká závislost.

Dejme tomu, že naše halda má n vrcholů v celkově h hladinách, přičemž poslední hladina je *plně obsazena* (viz obrázek 13). (Mohlo by se, v obecném případě, samozřejmě stát, že poslední hladina plně obsazena nebude, ale jediné, co se tím změní, bude, že mezi n a výsledným výrazem bude místo rovnosti nerovnost. S plnou hladinou se nám ale bude lépe počítat, protože při analýze časové složitost nás zajímá nejhorší možný případ.) Pokud budeme mít haldu s jednou hladinou, bude mít pouze jeden vrchol, a to kořen. S dvěma hladinami budou již 3 vrcholy v haldě. S třemi hladinami jich bude 7, atd (viz tabulka 1).

Počet hladin h	1	2	3	4	5	6
Počet vrcholů n	1	3	7	15	31	63

Tabulka 1: Vztah mezi počtem hladin h a počtem vrcholů n v plně obsazené haldě.



Obrázek 13: Odebírání vrcholu (kořene) s minimálním klíčem.

Z tabulky 1 si můžeme všimnout, že počet vrcholů je vždy *mocnina dvojky snižená o jedna*. Výsledná závislost je tedy $n = 2^h - 1$. V případě, že by poslední hladina nebyla plně obsazena, by se rovnost změnila v nerovnost, tj. $n \leq 2^h - 1$. Tím máme již vše připravené k tomu, abychom odvodili časovou složitost daných operací.

Věta 5.2 (Složitost operací v binární haldě). Operace INSERT, EXTRACTMIN, INCREASE a DECREASE mají časovou složitost $\mathcal{O}(\log n)$. Operace MIN má časovou složitost $\mathcal{O}(1)$.

Důkaz. V případě MIN je to jednoduché, zkrátka jen přečteme klíč v kořeni, což zjevně potrvá $\mathcal{O}(1)$. U zbývajících operací opravujeme haldu pomocí BUBBLEUP a BUBBLEDOWN. Předpokládejme, nastal nejhorší možný případ, tj. halda má h hladin a je plně obsazena (tzn. včetně poslední hladiny). Víme, že „probublání“ vrcholu haldou nahoru/dolů potrvá řádově $\mathcal{O}(h)$. My však víme, že $n = 2^h - 1$. Tedy zde bude stačit, když si vyjádříme h z rovnosti.

$$n = 2^h - 1 \iff 2^h = n + 1 \iff h = \log_2(n + 1) \stackrel{*}{=} \frac{1}{\log 2} \cdot \log(n + 1).$$

(V rovnosti $*$ jsme využili vzorec $\log_b a = \log a / \log b$.) Operace BUBBLEUP a BUBBLEDOWN tedy potrvají

$$\mathcal{O}(h) = \mathcal{O}\left(\frac{1}{\log 2} \cdot \log(n + 1)\right) \stackrel{*}{=} \mathcal{O}(\log n).$$

($V *$ je jednička v logaritmu zanedbatelná v rámci $\mathcal{O}$ -notace. Výraz $1/\log 2$ představuje konstantu a tudíž je též zanedbatelný.) Tedy operace INSERT, EXTRACTMIN, INCREASE a DECREASE mají logaritmickou časovou složitost. $\square$

Zkusme si na závěr udělat menší porovnání oproti poli, se kterým jsme začínali (viz tabulka 2). Logarit-

	INSERT	MIN	EXTRACTMIN	INCREASE	DECREASE
Pole (seznam)	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Binární halda	$\mathcal{O}(\log n)$	$\mathcal{O}(1)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$

Tabulka 2: Porovnání časových složitostí operací v haldě a v poli.

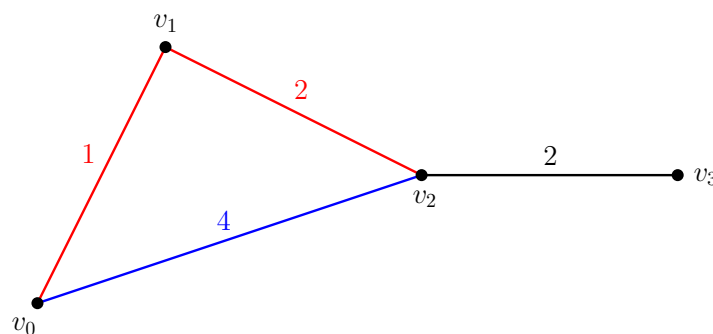
mická časová složitost je určitě lepší, než lineární, je však vidět, že jsme si zároveň pohoršili v případě vkládání a úpravy hodnot klíčů. Je tomu tak z důvodu, že pole má svou výhodu v téměř nulové režii, kdežto halda (kvůli své pevně definované struktuře) spotřebuje nějaký čas, aby byla uvedena do korektního stavu.

6 Dijkstrův algoritmus

Již jsme si ukázali, že v *neohodnoceném grafu* $G = (V, E)$ umíme relativně jednoduše najít délku nejkratší cesty do libovolného vrcholu z nějakého výchozího vrcholu $v_0 \in V$. Obvykle však hrany nemusí mít stejnou váhu (cenu). Např. cesty (vrcholy) mezi městy (hrany) mohou být v různém stavu a některé jsou tak lepší než jiné.



Zde ji narážíme na problém, neboť obecně platí, že když nalezneme vrchol přes dvojici různých hran, nemusí být nalezené cesty stejně dlouhé. Na obrázku 14 si lze všimnout, že cesta (v_0, v_1, v_2) je kratší než (v_0, v_2) , přestože má více hran.



Obrázek 14: Příklad ohodnoceného grafu.

Nabízí se varianta převést ohodnocený graf na neohodnocený tak, že rozdělíme hranu na takový počet (neohodnocených) hran, kolik činí její původní váha. V čistě teoretické rovině se jedná o funkční řešení, neboť na nově vzniklý graf již lze aplikovat např. BFS, které jsme popsali v sekci 3. Ne každý graf však musí mít „malé“ váhy hran. Pokud bychom vzali např. graf, kde váhy hran jsou v řádech tisíců, bude pro BFS potřeba provést zbytečně mnoho práce, neboť pro zpracování jedné ohodnocené hrany bude třeba provést tisíce iterací.

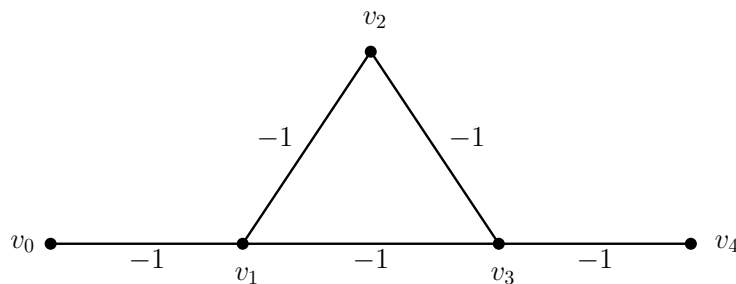
Jedním z nejznámějších algoritmů v tomto ohledu je tzv. *Dijkstrův algoritmus*, který popsal v roce

1958 holandský informatik *Edsger Wybe Dijkstra*.⁷ Podobně jako u BFS a DFS, i zde budeme postupně *otevírat* a *uzavírat* vrcholy, přičemž každý uzavřeme *nejvýše jednou*, avšak je třeba mít na paměti důležitou věc.



První nalezená cesta do libovolného vrcholu již nemusí být nutně nejkratší. Cestu do daného vrcholu bude třeba postupně vylepšovat.

Zaměříme se na grafy, jejichž váhy hran jsou nezáporné. Záporně ohodnocené hrany mohou způsobovat problémy, neboť mezi určitou dvojicí vrcholů by již nemusela nutně existovat nejkratší cesta. Např. v obrázku 15 si lze všimnout, že mezi vrcholy v_0 a v_4 neexistuje nejkratší cesta konečné délky, neboť cyklus (v_1, v_3, v_2, v_1) lze projít libovolněkrát, přičemž každým průchodem se zkrátí o 3. Odpověď na délku nejkrasší cesty mezi těmito vrcholy by tak musela být $-\infty$. Podobným grafům se tak chceme



Obrázek 15: Graf, kde mezi v_0 a v_4 neexistuje (konečná) nejkratší cesta.

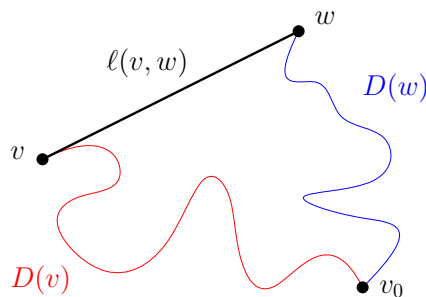
vyhnout, neboť by naše úvahy pak již nemusely fungovat (cestu mezi vrcholy bychom mohli potenciálně do nekonečně vylepšovat a algoritmus by se tak nikdy nezastavil).

V dalších odstavcích budeme váhu hrany vedoucí z vrcholu u do vrcholu v značit $\ell(u, v)$.

Algoritmus 6.1: DIJKSTRA

Vstup: Nezáporně ohodnocený graf $G = (V, E)$ a počáteční vrchol $v_0 \in V$
// Inicializace
1 **foreach** vrchol $v \in V$ **do**
2 $stav(v) \leftarrow \text{nenalezený}$
3 $D(v) \leftarrow \infty$
4 $stav(v_0) \leftarrow \text{otevřený}$
5 $D(v_0) \leftarrow 0$
6 **while** existují otevřené vrcholy **do**
7 // Kandidát na nejkrasší cestu do sousedních vrcholů
8 $v \leftarrow$ otevřený vrchol s nejmenším D
9 **foreach** soused $w \in V$ vrcholu v **do**
10 // Našli jsme lepší cestu
11 **if** $D(v) + \ell(v, w) < D(w)$ **then**
12 $D(w) \leftarrow D(v) + \ell(v, w)$
13 $stav(w) \leftarrow \text{otevřený}$
14 $stav(v) \leftarrow \text{uzavřený}$
Výstup: Seznam vzdáleností D

⁷Jedná se o holandské jméno, čteme „dajkstra“.



Obrázek 16: Znázornění situace v průběhu Dijkstrova algoritmu.

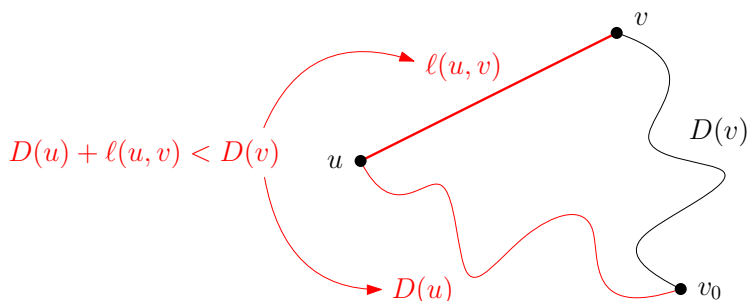
6.1 Správnost Dijkstrova algoritmu

Nejprve je dobré si uvědomit, proč algoritmus vlastně funguje. Kdykoliv uzavíráme vrchol (tzn. vybereme vrchol s nejmenší hodnotou $D(v)$), pak se spoléháme na to, že $D(v)$ opravdu odpovídá v daný moment délce nejkratší cesty z v_0 do v . Ale proč to tak musí být?

Zkusme uvážit opačnou situaci, tzn. kdyby nastalo, že jsme vybrali nějaký vrchol v , jehož $D(v)$ neodpovídá délce nejkratší cesty. Pak by musel existovat nějaký sousední vrchol u vrcholu v , přes nějž vede nejkratší cesta, jejíž délka je $D(u) + \ell(u, v)$ (viz obrázek 17). To znamená, že $D(v) > D(u) + \ell(u, v)$. Jenže v takovou chvíli by si algoritmus nutně musel vybrat vrchol u místo vrcholu v , protože

$$D(v) > D(u) + \ell(u, v) \stackrel{*}{>} D(u).$$

V kroku označeném $*$ vycházíme právě z toho, že váhy hran jsou nezáporné, tj. $\ell(u, v) > 0$ (proto tento argument funguje). Tedy stručně řečeno⁸, pokud $D(v)$ neodpovídá délce nejkratší cesty, pak musí existovat jiný vrchol u s nižším $D(u)$.



Obrázek 17: Situace při výběru vrcholu v , kdy $D(v)$ neodpovídá délce nejkratší cesty.

6.2 Časová složitost Dijkstrova algoritmu

Zbývá nám ještě analyzovat časovou složitost. Pokud se zpětně podíváme na pseudokód algoritmu, lze si všimnout, že podstatnou roli bude hrát vybírání vrcholu s minimální hodnotou $D(v)$. Nabízí se tak otázka, jak danou operaci implementovat. První variantou je hodnoty D realizovat jako prostý *seznam* (popř. *pole*).

Věta 6.1 (Dijkstra se seznamem). Dijkstrův algoritmus, kde hodnoty D ukládáme do seznamu (pole), doběhne v čase $\mathcal{O}(n^2 + m)$.

⁸Nebo spíš napsáno?

Důkaz. • Inicializace zabere $\mathcal{O}(n)$ (máme n vrcholů).

- Každý vrchol uzavřeme opět nejvýše jednou, tzn. vnější cyklus se provede $\mathcal{O}(n)$ -krát.
- Hledání minimálního D zabere v rámci každé iterace čas $\mathcal{O}(n)$.
- Stejně jako u BFS a DFS, i zde každou hranu zkontroluji nejvýše dvakrát (záleží, zda graf G je orientovaný), tzn. $\mathcal{O}(2m) = \mathcal{O}(m)$.

Celkově tedy máme

$$\overbrace{\mathcal{O}(n)}^{\text{Init}} + \underbrace{\mathcal{O}(n) \cdot \mathcal{O}(n)}_{\text{Výběr minima}} + \mathcal{O}(m) = \mathcal{O}(n^2 + n + m) = \mathcal{O}(n^2 + m).$$

□

Kvadratická časová složitost není úplně zlá, ale můžeme si ještě trochu přilepšit. Pokud si vzpomeneme na binární haldy z oddílu 5, všimneme si v konečném důsledku, že při použití v Dijkstrově algoritmu se některé operace zrychlí.

Věta 6.2 (Dijkstra s haldou). Dijkstrův algoritmus, kde hodnoty D ukládáme do binární haldy, doběhne v čase $\mathcal{O}((n + m) \cdot \log n)$.

Důkaz. Inicializace trvá zase $\mathcal{O}(n)$. Při operacích s binární haldou víme, jak dlouho potrvají (viz tabulka 2). Akorát si nyní musíme rozmyslet, kdy se která operace provádí.

- Při *otevírání* vrcholu provádíme v haldě operaci INSERT⁹. Celkově vložíme do haldy max. všech n vrcholů, což potrvá $n \cdot \mathcal{O}(\log n) = \mathcal{O}(n \log n)$.
- Při *uzavírání* vrcholu provádíme operaci EXTRACTMIN (opět max. n -krát), tj. $n \cdot \mathcal{O}(\log n) = \mathcal{O}(n \log n)$.
- Při *aktualizaci vzdálenosti* $D(v)$ provádíme DECREASE (tato hodnota se nikdy nemůže zvýšit). To provádíme vždy při kontrole sousedů (tzn. daných hran), kterých je pro všechny vrcholy dohromady max. $\mathcal{O}(m)$ (stejný argument jako při započítávání hran). Tzn. $m \cdot \mathcal{O}(\log n) = \mathcal{O}(m \log n)$.

Po sečtení dostaneme

$$\begin{aligned} \mathcal{O}(n + n \log n + n \log n + m \log n) &= \mathcal{O}(n + 2n \log n + m \log n) \\ &= \mathcal{O}(n + n \log n + m \log n) \\ &= \mathcal{O}(n \log n + m \log n) \\ &= \mathcal{O}((n + m) \cdot \log n). \end{aligned}$$

□

Ačkoliv to není úplně zjevné, výraz $(n + m) \cdot \log n$ roste o dost pomaleji oproti $n^2 + m$, tedy binární haldou si skutečně pomůžeme oproti seznamu.

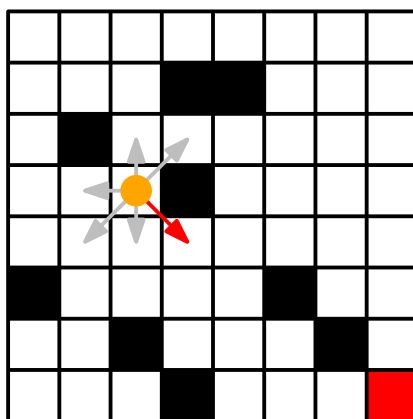
Závěrem zde je ještě uvedme, že často nehledáme cestu do všech vrcholů, ale pouze mezi nějakou konkrétní dvojicí. V takovou chvíli můžeme výpočet algoritmu zarazit ve chvíli, kdy narazíme na cílový vrchol, protože jak již víme, při otevírání vrcholu je $D(v)$ skutečná délka nejkratší cesty. V tomto případě je možné ještě dále urychlit hledání nejkratší cesty, neboť Dijkstrův algoritmus může pak provádět hodně nadbytečné práce. Na to se podíváme v další sekci 7.

⁹Náš odhad $\mathcal{O}(n \log n)$ by šel dokonce zlepšit i na $\mathcal{O}(n)$, když do něj započítáme fakt, že hladin v haldě je ze začátku 0 a postupně přibývají (tzn. není jich z počátku „moc“), ale celkový odhad složitosti by vyšel stejně a navíc odvození tohoto faktu je již trochu náročnější.

7 Algoritmus A*

Nyní omezíme naše soustředění na případy, kdy hledáme cestu pouze do nějakého konkrétního vrcholu¹⁰ $v_G \in V$ (nikoliv do všech, jako tomu bylo doposud). Stále pracujeme s grafy, jejichž hrany mají nezáporné ohodnocení. Všechny dosavadně vysvětlené algoritmy by jistě fungovaly i v tomto případě (při otevírání cílového vrcholu jednoduše vyskočíme z hlavního cyklu). Nabízí se však otázka, zdali nemůžeme znalosti cílového vrcholu nějak využít. Pokud bychom např. hledali nejkratší cestu v mřížce s rozmístěnými překážkami (tj. na některá políčka nelze vstoupit), nejpíše by dávalo smysl upřednostňovat políčka, která jsou blíže cílovému políčku, než ty, která jsou od něj dál.

Mějme bludiště, v němž se nachází hráč (oranžový kruh), jež se může pohybovat libovolným směrem vždy o jedno pole, přičemž cílovým pole se nachází vpravo dole (červeně vyznačené). Situaci lze vidět na obrázku 18. Takovou situaci můžeme znázornit pomocí grafu, kde každá hrana má váhu 1, popř. lze použít neohodnocený graf. Pokud bychom aplikovali např. BFS, či Dijkstrův algoritmus, prozkoumali bychom



Obrázek 18: Mřížka a optimální směr pohybu hráče.

postupně všechny směry (šipky) stejně. Je však zřejmé, že nejvýhodnější je vydat směrem vyznačeným červeně, neboť je nejblíže cíli. V roce 1968 tak přišli Peter Hart, Nils Nilsson a Bertam Raphael s úpravou původního Dijkstrova algoritmu, která nese název A*.

Myšlenka je stále stejná, avšak nyní budeme vybírat vrcholy podle hodnoty $D(v) + \psi(v)$ (místo pouhého $D(v)$), kde ψ je tzv. *heuristická funkce* nebo zkráceně *heuristika*, která slouží jako *odhad vzdálenosti od cílového vrcholu* v grafu (viz obrázek 19). Součtu $D(v) + \psi(v)$ budeme říkat tzv. *f-skóre*¹¹, značíme $f(v)$. V každé iteraci tak budeme vybírat vrchol s nejnižším *f-skóre*, k čemuž lze opět použít pole, nebo binární haldy.

Oproti Dijkstrově algoritmu tedy skutečně nenastává moc změn, akorát je potřeba si zvlášť uchovávat hodnoty *f-skóre*.

7.1 Správnost algoritmu A*

Podobně jako u Dijkstrova algoritmu i zde platí, že při výběru (tj. otevírání) vrcholu v odpovídá $D(v)$ délce nejkratší cesty. Argument pro tuto skutečnost je podobný. Předpokládejme, že otevíráme nějaký vrchol v , přičemž hodnota $D(v)$ je větší, než je skutečná délka nejkratší cesty. Pak existuje nějaký

¹⁰ „G“ od angl. *goal*

¹¹ V jiných textech, převážně anglických, se někdy používá pro heuristiku označení h a pro vzdálenost označení g , tzv. *g-skóre*, tj. $f(v) = g(v) + h(v)$.



nespecifikovali žádné vlastnosti, které bychom od ní očekávali. Ukazuje se však, že ψ si libovolně zvolit nemůžeme. Naše heuristika musí být v jistém smyslu „rozumná“. Pokud se vydáme z vrcholu u po hraně do vrcholu v a z něj poté do cílového vrcholu v_G , určitě výsledná cesta bude alespoň tak dlouhá, jako nejkratší cesta z u do v_G . To samé by mělo platit i pro náš odhad vzdálenosti¹², tedy $\psi(u) \leq \ell(u, v) + \psi(v)$.

$$f(u) = D(u) + \overbrace{\psi(u)}^{\leq \ell(u,v) + \psi(v)} \leq \underbrace{D(u) + \ell(u,v)}_{< D(v)} + \psi(v) < D(v) + \psi(v) = f(v).$$

Upozorníme zde na fakt, že náš předpoklad o heuristice je velmi důležitý, neboť pokud bychom si zvolili heuristiku nevhodně, algoritmus již fungovat nebude a obecně může dojít k tomu, že nějaký z vrcholů vybereme „předčasně“.

18

Algoritmus 7.1: A*

Vstup: Nezáporně ohodnocený graf $G = (V, E)$ a počáteční vrchol $v_0 \in V$
// Inicializace
1 **foreach** vrchol $v \in V$ **do**
2 $stav(v) \leftarrow \text{nenalezený}$
3 $D(v) \leftarrow \infty, f(v) \leftarrow \infty$
4 $stav(v_0) \leftarrow \text{otevřený}$
5 $D(v_0) \leftarrow 0, f(v_0) \leftarrow \psi(v_0)$
6 **while** existují otevřené vrcholy **do**
7 $v \leftarrow$ otevřený vrchol s nejmenším f -skóre
8 **if** $v = v_G$ **then**
9 **return** $D(v_G)$
10 **foreach** soused w vrcholu v **do**
11 // Našli jsme lepší cestu
12 **if** $D(v) + \ell(v, w) < D(w)$ **then**
13 $D(w) \leftarrow D(v) + \ell(v, w)$
14 $f(w) \leftarrow D(v) + \ell(v, w) + \psi(w)$
15 $stav(w) \leftarrow \text{otevřený}$
16 $stav(v) \leftarrow \text{uzavřený}$
Výstup: Vzdálenost v_0 od v_G

Poznámka 7.1. • Všimněme si, že Dijkstrův algoritmus je speciálním případem algoritmu A*. Stačí pro všechny vrcholy v položit¹³ $\psi(v) = 0$.

- Je dobré dodat, že heuristiku bychom měli být schopni pro každý vrchol spočítat co nejrychleji, nejlépe v konstantním čase, tj. $\mathcal{O}(1)$.

Rozbor časové složitosti zde vynecháme. Je totiž silně závislá na tom, jakou heuristiku ψ si zvolíme. Při vhodné volbě doběhne A* zpravidla rychleji oproti Dijkstrově algoritmu. Může se však také stát, že ψ zvolíme nevhodně (i přesto, že splňuje trojúhelníkovou nerovnost výše), a algoritmus nakonec poběží déle.

7.2 Ukázky heuristických funkcí

Zbývá zodpovědět otázku, jakou heuristiku tedy použít? Odpověď není zcela jednoznačná, neboť záleží na situaci. Vraťme se opět k příkladu s bludištěm, kdy začínáme na určitém políčku a snažíme se dostat do cílového, přičemž hráč se smí pohybovat všemi směry vždy o jedno pole. Políčka identifikujeme podle jejich souřadnic, tj. dvojice (x, y) .

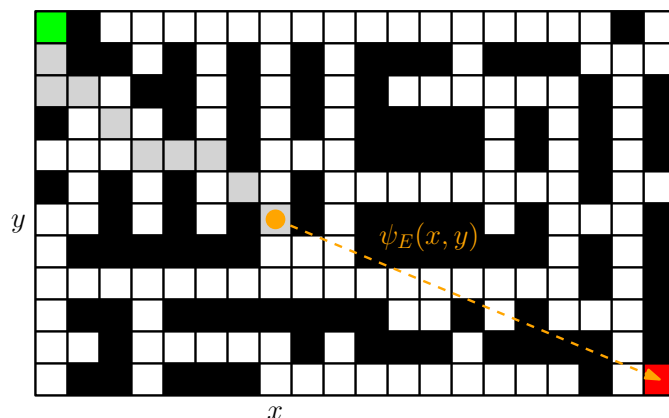
Jednou možností je tzv. *eukleidovská vzdálenost*. Ta je pro libovolnou dvojici bodů v rovině $X = (x_1, y_1)$ a $Y = (x_2, y_2)$ definována jako

$$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}.$$

V případě cílového políčka známe jeho souřadnice, takže můžeme snadno vypočítat eukleidovskou vzdálenost libovolného políčka od cílového. To lze použít jako heuristiku¹⁴, označme si ji ψ_E . Situaci lze sledovat na obrázku 21, kde šedě je vyznačena nejkratší cesta do pole se souřadnicemi (x, y) . Pokud

¹³Lze zvolit klidně i obecnou konstantu $c \in \mathbb{R}$, tj. $\psi(v) = c$.

¹⁴Dokonce bychom mohli použít jako heuristiku pouze výraz pod odmocninou, tj. $(x_1 - x_2)^2 + (y_1 - y_2)^2$.

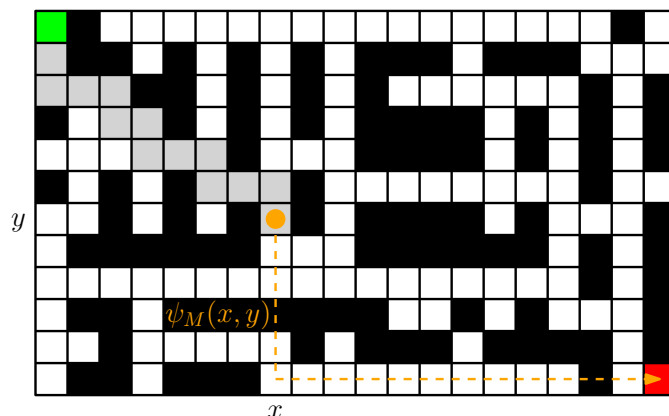


Obrázek 21: Příklad bludiště s eukleidovskou heuristikou.

bychom však pracovali s bludištěm, kde se *nelze pohybovat po diagonálách* nebo by nám to pravidla neumožňovala, můžeme výpočet heuristické funkce nahradit něčím jednodušším. V tomto ohledu se pak často využívá tzv. *manhattanská vzdálenost* (označme ψ_M) definovaná vztahem

$$|x_1 - x_2| + |y_1 - y_2|.$$

Stejně jako eukleidovská vzdálenost, i manhattanská vzdálenost splňuje trojúhelníkovou nerovnost.



Obrázek 22: Příklad bludiště s manhattnskou heuristikou.

Její výhodou je nižší náročnost při výpočtu (výpočet odmocniny¹⁵ je již trochu pomalejší) a to hlavně z důvodu, že pracujeme vždy s celými čísly. Její nevýhodou však může být, že oproti eukleidovské vzdálenosti nabývá větších hodnot.

¹⁵V mnoha případech, např. v některých starších videohráčích jako Quake III, se vývojáři snažili vyhýbat výpočtu odmocniny, nebo volili některé sofistikovanější výpočty, které byly rychlé a poskytovaly uspokojivý odhad hledané hodnoty.